

TECHNICAL PUBLICATION · PERSONAL RESEARCH

Transformers Revisited:

Architecture, Attention, and Limitations

*A progressive and conceptual analysis of the foundations
and recent advances in sequence modeling*

Narcisse K. ATTIOU

2026

Abstract

Transformers are now the dominant architecture in modern artificial intelligence. Introduced in 2017 by Vaswani et al. in the context of natural language processing, they have since revolutionized computer vision, music generation, bioinformatics, and many other fields. This paper presents a progressive and conceptual analysis of their architecture from the attention mechanism to modern positional encodings (RoPE, ALiBi) along with advanced training methods (RLHF, instruction tuning) and recent optimizations (FlashAttention, sparse attention). Fundamental limitations and emerging alternatives (state space models, Mamba) are also examined to provide a complete and up-to-date picture of the sequence modeling landscape.

Keywords: transformers, self-attention, BERT, GPT, RoPE, FlashAttention, RLHF, foundation models, NLP, deep learning.

1. Introduction and Context

Since their introduction in 2017 [1], transformers have progressively superseded recurrent (RNN, LSTM) and convolutional (CNN) architectures across virtually all sequence modeling tasks. This rise is not accidental: it is built on fundamental architectural choices that directly address the structural limitations of their predecessors.

Traditional sequential models suffered from two major problems:

- **Long-range forgetting:** distant information in a sequence was degraded over recurrence steps, making it difficult to capture semantic dependencies between remote elements.
- **Non-parallelizability:** step-by-step sequential computation prevented efficient use of modern GPUs.

Transformers solve both problems simultaneously through the attention mechanism: the entire sequence is processed in parallel, and each element can directly interact with every other, regardless of distance.

2. Philosophy and Paradigm

The philosophy of transformers rests on a radical shift in perspective: a sequence is no longer treated as a temporal chain, but as an unordered set of tokens whose relationships are learned dynamically. This implies:

- A **global, non-local** view of the sequence;
- A context built **dynamically** for each token, rather than propagated sequentially;

- **Modularity and scalability** that facilitate scaling to billions of parameters.

This paradigm shift had cascading effects on the entire field. Scaling laws (Kaplan et al., 2020 [10]) showed that the performance of transformer based models grows predictably with the number of parameters, data volume, and compute budget paving the way for Foundation Models.

3. Detailed Architecture

3.1 Embeddings and Token Representation

Each text token is mapped to a dense vector of dimension d_{model} (typically 512, 768, or 1024) via an embedding layer. These vectors are randomly initialized and learned during training. They capture semantic similarities: words that are close in vector space share semantic or syntactic properties.

For multimodal models such as ViT [5], image patches play the role of text tokens: each region of the image is projected into the same vector space as text tokens.

3.2 Positional Encoding

Transformers have no intrinsic notion of order. Without positional encoding, the model would treat a bag of tokens with no structure. Three main families of solutions exist:

Sinusoidal (Vaswani et al., 2017). Deterministic signal based on sine/cosine functions of varying frequencies. Requires no training and generalizes to sequences longer than those seen during training.

Learned Positional Embeddings. Positions are encoded as learnable parameters. Used by BERT and GPT-2. More expressive but limited to the maximum sequence length seen during training.

RoPE: Rotary Position Embedding [7]. Encodes position in the key/query space via complex rotation. Key advantage: preserves relative relationships between tokens. Used by LLaMA, GPT-NeoX, Mistral, and Gemma.

ALiBi :Attention with Linear Biases. Adds a linearly decreasing bias to attention scores as a function of distance. Highly effective for out of distribution length generalization.

3.3 The Attention Mechanism

The attention mechanism is the core of the transformer. Each token is simultaneously projected into three vector spaces via learned weight matrices:

- **Query (Q):** what the token is looking for in the context;
- **Key (K):** what the token exposes to other tokens;
- **Value (V):** the effective information transmitted if the token is selected.

The attention score between tokens i and j is the dot product of query i and key j , scaled by $\sqrt{d_k}$ to stabilize gradients:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (3.1)$$

Intuitively, the model learns to build a contextualized representation for each token, weighted by the relevance of every other token. This operation is fully parallelizable.

Concrete example. In the sentence *"The bank rejected the loan because it was bankrupt"*, the pronoun *"it"* must refer to *"bank"* and not to *"loan"*. The attention mechanism allows the model to learn this coreference directly.

3.4 Multi-Head Attention

Rather than learning a single set of Q/K/V projections, the model trains h attention heads in parallel, each with its own projection matrices. Each head specializes in a different type of relationship (syntax, coreference, semantics, etc.). Representations from all heads are concatenated and projected:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O \quad (3.2)$$

$$\text{head}_i = \text{Attention}(QW_{Q_i}, KW_{K_i}, VW_{V_i}) \quad (3.3)$$

The typical number of heads is 8 or 16, with $d_k = d_{\text{model}}/h$ to keep computational cost constant.

3.5 Position-Wise Feed-Forward Networks

After attention, each token is independently transformed (position by position) through a two-layer feed-forward network with a non-linear activation:

$$\text{FFN}(x) = \text{Linear}_2(\text{GELU}(\text{Linear}_1(x))) \quad (3.4)$$

The intermediate dimension is typically $d_{\text{ff}} = 4 \times d_{\text{model}}$. Modern models like LLaMA use **SwiGLU** instead of GELU for better performance. This component acts as a learned associative memory for storing factual knowledge.

3.6 Residual Connections and Layer Normalization

Each sub-block (attention + feed-forward) is wrapped with a residual connection and a layer normalization. Two orderings are possible:

- **Post LN** (original transformer): $x \rightarrow \text{SubLayer} \rightarrow \text{Add} \rightarrow \text{LayerNorm}$
- **Pre LN** (modern standard): $x \rightarrow \text{LayerNorm} \rightarrow \text{SubLayer} \rightarrow \text{Add}$ — better training stability.

These mechanisms are essential for training deep models (up to 96+ layers): residual connections allow gradients to flow directly, and LayerNorm normalizes activations to prevent exploding or vanishing gradients.

3.7 Encoder and Decoder

The standard architecture distinguishes two roles:

- **Encoder (BERT [2]):** processes the input sequence with bidirectional attention. Optimal for comprehension and classification tasks.
- **Decoder (GPT [3]):** generates tokens one by one with masked causal attention. Optimal for text generation.
- **Encoder-Decoder (T5 [4]):** combines both. The encoder processes the source; the decoder generates the target with cross-attention over encoder representations.

4. Transformer-Based Models

The transformer architecture has produced a rich family of models across very diverse domains:

Table 1: Overview of major transformer-based models

lightblue Model	Description
BERT [2]	Bidirectional encoder pre-trained with Masked Language Modeling (MLM). Reference for comprehension tasks (classification, QA, NER).
GPT-3/4 [3]	Large-scale autoregressive decoder. Emergent capabilities (few-shot learning) appear at 10B+ parameters.
T5 [4]	Encoder-decoder unifying all NLP tasks under a text-to-text format.
ViT [5]	Transformers applied to vision: image split into patches treated as tokens.
LLaMA / Mistral	Open modern decoders using RoPE, Grouped Query Attention (GQA), and SwiGLU. Highly efficient.
Multimodal (GPT-4V, Gemini)	Simultaneously process text, images, and audio via specialized encoders.

5. Modern Training Methods

5.1 Self Supervised Pre-training

Transformers are first pre-trained on massive corpora (hundreds of billions to several trillion tokens) via self supervised objectives that require no human annotation:

- **Masked Language Modeling (MLM) BERT:** 15% of tokens are masked; the model predicts the missing tokens.
- **Causal Language Modeling (CLM) GPT:** the model predicts the next token given all previous ones (autoregressive prediction).
- **Span Corruption T5:** spans of tokens are replaced by sentinel tokens, and the model reconstructs the missing spans.

5.2 Supervised Fine-Tuning

After pre-training, the model is adapted to specific tasks via fine-tuning on labeled data. Modern techniques include:

- **Full fine-tuning:** all parameters updated. Costly but performant.
- **LoRA (Low-Rank Adaptation):** low rank matrices injected into attention layers. Reduces the number of trainable parameters by 90%+ while maintaining performance.
- **Prompt Tuning / Prefix Tuning:** optimization of virtual context tokens, without modifying model weights.

5.3 Instruction Tuning and RLHF

Modern language models (ChatGPT, Claude, Gemini) integrate an alignment phase after pre-training:

Instruction Tuning. Fine-tuning on thousands of (instruction, ideal response) pairs covering diverse tasks. The model learns to follow natural language instructions rather than simply completing text.

RLHF Reinforcement Learning from Human Feedback [9]. A reward model is trained from human preferences between multiple responses, then used to refine the LLM via PPO (Proximal Policy Optimization). Significantly improves safety, helpfulness, and honesty.

DPO: Direct Preference Optimization. A more stable alternative to RLHF: directly optimizes preferences without an explicit reward model. Used by many open-source models.

6. Complexity, Limitations, and Optimizations

6.1 Quadratic Complexity of Attention

The fundamental limitation of transformers is the quadratic complexity of attention as a function of sequence length n :

$$\text{Complexity: } \mathcal{O}(n^2 \cdot d) \quad \text{Memory: } \mathcal{O}(n^2) \quad (6.1)$$

For $n = 1,000$ tokens, the attention matrix contains one million values. For $n = 100,000$ (long documents, genomes, full codebases), this becomes prohibitive. This constraint has motivated a wave of research on efficient attention.

6.2 FlashAttention — Memory Optimization

FlashAttention (Dao et al., 2022 [6]) is an algorithmic reformulation of attention that exploits the GPU memory hierarchy (SRAM vs. HBM). By splitting matrices into blocks (*tiling*) and fusing operations, FlashAttention reduces memory from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$ and speeds up computation by 2–4×, with no loss in precision.

FlashAttention is now integrated as a standard component in PyTorch and is used by virtually all large models: LLaMA, GPT-4, Claude, Gemini. It is one of the most impactful practical contributions of the past five years.

6.3 Other Efficient Attention Approaches

- **Sparse Attention: Longformer [8]:** each token attends only to a local neighborhood + a few global tokens. Complexity $\mathcal{O}(n \cdot k)$.
- **Linear Attention: Performer [11]:** approximates softmax via random features. Complexity $\mathcal{O}(n)$ but approximate.
- **Sliding Window + Global Tokens (Mistral, Gemma):** sliding window with global tokens for key passages.

6.4 Alternatives to Transformers

Quadratic complexity has motivated exploration of alternative architectures:

- **Mamba / State Space Models (SSMs) [12]:** selective state space models offering $\mathcal{O}(n)$ complexity with performance comparable to transformers on many tasks.
- **Hybrid models (Jamba, Zamba):** combining attention layers with SSM layers, benefiting from the strengths of both approaches.

7. Architecture Comparison

Table 2: Comparison of sequence modeling architectures

lightblue Architecture	Long-range deps.	Parallelization	Complexity	Memory
RNN / LSTM	Low	Low	$\mathcal{O}(n)$	Low
CNN	Medium	High	$\mathcal{O}(n \cdot k)$	Medium
Transformer	Very High	Very High	$\mathcal{O}(n^2)$	High
Sparse / Flash Attn	Very High	Very High	$\mathcal{O}(n \log n)$	Reduced
Mamba (SSM)	High	High	$\mathcal{O}(n)$	Low

8. Pseudo-Code and Implementation

Listing 1: General structure of a transformer forward pass (Pre-LN variant)

```

1 def transformer_forward(tokens):
2     # Embedding + positional encoding
3     x = embedding(tokens) + positional_encoding(tokens)
4
5     for layer in transformer_layers:
6         # --- Self-Attention block ---
7         residual = x
8         x = layer_norm(x)
9         attn_out = multi_head_attention(x) # Eq. (3.1)
10        x = residual + dropout(attn_out)
11

```

```

12     # --- Feed-Forward block ---
13     residual = x
14     x = layer_norm(x)
15     ffn_out = feed_forward(x)           # Eq. (3.3)
16     x = residual + dropout(ffn_out)
17
18     # Final projection
19     return linear_projection(layer_norm(x))

```

Listing 2: Scaled dot-product attention

```

1  import torch
2  import torch.nn.functional as F
3  import math
4
5  def scaled_dot_product_attention(Q, K, V, mask=None):
6      d_k = Q.size(-1)
7      # Compute attention scores
8      scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
9      # Apply optional causal mask (decoder)
10     if mask is not None:
11         scores = scores.masked_fill(mask == 0, float('-inf'))
12     weights = F.softmax(scores, dim=-1)
13     return torch.matmul(weights, V), weights

```

9. Conclusion

Transformers have fundamentally reshaped the landscape of artificial intelligence. By enabling direct modeling of long range dependencies and massive parallelization, they made possible the emergence of foundation models capable of extraordinary performance across a broad range of tasks.

Recent developments RoPE for positional encoding, FlashAttention for memory efficiency, RLHF for alignment, LoRA for efficient adaptation demonstrate that the architecture is not static: it continues to evolve rapidly. Alternatives such as State Space Models (Mamba) further enrich the landscape.

A deep understanding of these mechanisms is now indispensable for any AI practitioner. Beyond the technical aspects, transformers also raise fundamental questions about the nature of representation and artificial reasoning questions that remain largely open and constitute an extraordinarily fertile field of research.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT*, 2019.
- [3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.

- [4] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research (JMLR)*, 2020.
- [5] A. Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.
- [6] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.
- [7] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv:2104.09864*, 2021.
- [8] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. *arXiv:2004.05150*, 2020.
- [9] L. Ouyang et al. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022.
- [10] J. Kaplan et al. Scaling laws for neural language models. *arXiv:2001.08361*, 2020.
- [11] K. Choromanski et al. Rethinking attention with performers. In *ICLR*, 2021.
- [12] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*, 2023.

Written by **Narcisse K. ATTIOU** · 2026 · Personal technical publication